Choose Your Own Project: Handwritten Character Recognition

edX HarvardX: PH125.9x - Data Science: Capstone

Azamat Kurbanaev

2026-11-11

Contents

Preface 4

Acknowledgements 5

1 Overview and Executive Summary 6

1.1 Project Description 6

1.2 Environment Setup 8

1.2.1 Batch Execution & Cleanup Scripts 8

1.3 Running the Project 9

1.3.1 The Main R Script 10

1.3.1.1 Setup & Initialization 10

1.3.1.2 Downloading & Preprocessing Input Datasets . . 11

1.3.1.3 Machine Learning Model Pipelines 12

1.4 Datasets Overview 14

2 Methods and Analysis 18

2.1 kNN+PCA MCC Model 18

2.2 Random Forest (RF) MCC Model 18

2.3 Dense Neural Network Based Multiclass Classifier (DNN-Based MCC) Models 18

2.3.1 DNN-Based Basic (DNNB) MCC Model 18

2.4 Tuned DNN-Based (TDNN) MCC Model 18

2.5 Convolutional Neural Network Based (CNN) MCC Models 18

2.5.1 CNN-Based Basic (CNNB) MCC Model 18

2.6 Tuned CNN-Based (TCNN) MCC Model 18
2.6.1 TCNN MCC: Final Test 18

3 Results **19**
3.1 The Most Confusing Characters for Recognition 19

4 Conclusion **20**

5 Appendix A: Support Functions **21**
5.1 Data Helper Script 21
5.1.1 Data Loading 21
5.1.1.1 kaggle_cli.download Function 21
5.1.1.2 load_image.list Function 26
5.1.1.3 img.file.get_list Function 37
5.1.1.4 load.img_trimmed.dat Function 45
5.1.2 Image Processing 48
5.1.2.1 magick_img2matrix Function 48
5.2 Models Helper 52
5.2.1 Calculating PCA & Constructing Bar Chart 52
5.2.1.1 MCClassifier.accuracy.by_class Function 52

6 Appendix B: Support Scripts **54**
6.1 r-global.path.R Script 54
6.2 setup.R Script 54
6.2.1 Install Packages Section 55
6.2.2 Load Libraries & resolve conflicts Section 55
6.2.3 Load Logging Helper Functions Section 55
6.2.4 Init Project Directories Section 55
6.2.5 Init Input Data Paths Section 55
6.2.6 Init Project Script Paths Section 55
6.2.7 Load Common Helper Functions Section 55
6.2.8 Load Data Helper Functions Section 56
6.2.9 Load Model Helper Functions Section 56

6.2.10 Load `CNN_MCC.HyperModel` helper class Section 56
6.2.11 Load `CNN_MCC.HyperParameters` helper class Section . . 56
6.3 ds.kaggle.download.R Script 56
6.4 load-raw-image-data.R Script 56
6.5 raw-img.dat2std_bin.R Script 57
6.6 prepare-training-set.R Script 57
6.7 Script 57
6.8 Script 57
6.9 Script 57
6.10 Script 57

7 Appendix C: Device (Laptop) Used for Development & Testing 58
7.1 Ninkear Laptop 58
7.1.1 System 58
7.1.1.1 Device Specifications 58
7.1.1.2 Windows Specifications 58

Preface

This project is the second assignment of the 'Data Science: Capstone' course (PH125.9x), offered by edX HarvardX. The project aims to build and compare different Handwritten Character Recognition (HCR) models, applying different Machine Learning (ML) techniques, and using a publicly available dataset for training and validation.

Acknowledgements

I would like to extend my deepest gratitude to Professor **Rafael A. Irizarry** for designing and delivering the comprehensive courses within the ***Professional Certificate Program in Data Science***. Prof. **Irizarry**'s instruction has provided me with the essential theoretical foundations and practical toolkit required for modern data analysis.

I am equally grateful for his textbook, *Introduction to Data Science*, (part 1[1] and part 2[2]). This text served as an indispensable guide throughout my coursework, offering clear explanations and robust examples that bridged the gap between theory and execution. Prof. **Irizarry**'s dedication to creating clear, accessible educational material has profoundly enhanced my learning experience and will undoubtedly continue to serve as a key reference in my future endeavors.

Chapter 1

Overview and Executive Summary

As noted in the article Handwritten Character Recognition: A Comprehensive Survey, Handwritten Character Recognition (HCR) plays a vital role in numerous domains, including document digitization, automatic text recognition, signature verification, and postal automation. Over the years, HCR has advanced significantly, driven by the growing demand for accurate, efficient recognition systems [3].

1.1 Project Description



All the source files for *this Project* are available in the *Project*'s GitHub repository[4].

In this project, we will develop and explore several HCR models, based on popular ML algorithms that convert handwritten alphanumeric symbols into a digital, machine-readable form.

We will start with Shallow Learning (SL) models, using k-Nearest Neighbors (kNN) and Random Forest (RF) methods. These models development and evaluation are described in detail below in Sections [k-Nearest Neighbors + Principal Component Analysis (kNN+PCA) Multiclass Classifier (MCC) Model] and Random Forest (RF) MCC Model of *this Report*, respectively.

Next, we will try Deep Learning (DL) models, including Dense Neural Network (DNN) and, finally, Convolutional Neural Network (CNN) models, as recent re-

search shows that DL performs better at image recognition — the task addressed by *this Project*.

A more detailed explanation of *Deep Learning* versus *Shallow Learning* can be found in the chapter Machine Learning Methods from Shallow Learning to Deep Learning of the book Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions by editors Ömer Faruk Ertuğrul, Josep M. Guerrero, and Musa Yilmaz[5].

The ***DNN-Based*** models are as follows:

- ***DNN-Based Basic MCC Model:*** described in Section [Dense Neural Network-Based Basic Multiclass Classifier (DNNB MCC) Model];
- ***Tuned DNN-Based MCC Model:*** described in Section Tuned DNN-Based (TDNN) MCC Model.

The last models we will build and explore, the ***CNN-Based*** models, are as follows:

- ***CNN-Based Basic MCC Model:*** described in Section [Convolutional Neural Network-Based Basic (CNNB) MCC Model];
- ***Tuned CNN-Based MCC Model:*** described in Section Tuned CNN-Based (TCNN) MCC Model.

Finally, we will conduct a final test of the best model we have developed, using a specially designated dataset, and see what we will achieve.

1.2 Environment Setup



The environment setup instructions are provided in the Prerequisites & Setup Section of the README document on GitHub.

The environment setup configures an `R`-based machine learning environment with *Python* integration, required to build the *Deep Learning* models.

The installation follows a *4-step* process by sequentially running the following scripts:

- 1.install-reticulate&miniconda.R:
  1. *Installs reticulate:* Configures the primary `R` package, the R Interface to Python;
  2. *Installs Miniconda* to manage Python virtual environments cleanly without conflicting with system Python installations.

- 2.install-tensorflow.R:
  1. **Installs the R TensorFlow interface:** Loads the TensorFlow R bindings;
  2. **Configures Backend Engine:** Uses reticulate to set up a dedicated *Python virtual/conda environment* and installs the core C++/Python TensorFlow package inside it.

- 3.install-keras3.R:
  1. **Installs the Keras3 package:** Installs the R Interface for Keras3.
  2. **Binds High-Level Neural Network APIs:** Integrates `Keras` on top of the backend installed in step 2 to enable building and training *Deep Learning* models in `R`.

- 4,install-kerastuner.R: Installs ***Hyperparameter Tuning Tools***, including R interface to Keras Tuner. Configures the KerasTuner in `R` to automate Hyperparameter Optimization (*search space*, *trial execution*, and *model evaluation*).

1.2.1 Batch Execution & Cleanup Scripts

As stated in the Prerequisites & Setup Section of the README document on GitHub, on *Windows OS*, all 4 steps of the setup process described above can be completed automatically by launching the ___setup-env.cmd *Windows command script* in *PowerShell* or *Command Prompt*.

The script ___uninstall-all.R cleans up the environment by wiping installed packages, *Conda environments*, and *Python bindings*.

1.3 Running the Project

Once the system environment is fully configured, if you have enough time to execute all the scripts and automatically generate *this report* upon their completion in a single session, the easiest approach is to run the ___run-all.R script.

This script serves as a starting point for running the project execution pipeline that culminates in a *PDF report*. It coordinates the process by sequentially executing two main R scripts, ensuring the script files exist, and outputs detailed execution logs to both the console and log files.

The first script it launches, index.R, is the entry point to the project's execution pipeline and the main script that runs all the other execution scripts sequentially.

The second (and last) script it runs, ___rmd.render.R, renders *this Report*.



Some scripts may take a long time when executed on their first run. In particular, the longest run on the *author's laptop* (for technical specifications, see Appendix C) was the first run of the 2.cnn-mcc.lr-tuner.R script (for fine-tuning the *CNN-based MCC* model) and took about **17 hours**.

The other long-running scripts are as follows:

- For tuning and retraining the best (and *final*) *CNN-based MCC* model:
  - 1.cnn-mcc.arch-tuner.R (took more than **6 hours** on the author's laptop);
  - 3.tcnn-mcc.final.retrain.R (took around **6.5 hours**);
- For tuning, retraining, and evaluating the best *kNN+PCA MCC* model:
  - 3.knn+pca.best-eval.R (took more than **2 hours**);
  - 2.knn+pca.re-train.best-k.R (took about **2 hours**);
  - 1.knn+pca.build&tune.R (about *half an hour*).
- For tuning and retraining the best *Random Forest (RF) MCC* model:
  - 2.rf-retraining.best-par.R (more than **1.5 hours**).
  - 1.rf-tuning.R (more than ***an hour***).
- For tuning the *DNN-Based MCC* model:
  - 1.dnn-mcc.tuner.R (more than *half an hour*).

- For downloading, analyzing, and preprocessing datasets:
  - ds.kaggle.download.R (about ***an hour***): downloads the Kaggle dataset;
  - load-raw-image-data.R (for the *Training Set*, around *40 minutes*): loads into memory the *raw image data* previously downloaded from Kaggle.
  - raw-img.dat2std_bin.R (for the *Training Set*, about *half an hour*): prepares *standardized binary* image data;



Subsequent runs of these long-running scripts will be much faster thanks to a caching mechanism that checks for the existence of the pre-computed results before running time-consuming training or evaluation code and loads the results from the cache files (if any). Besides, any of these scripts can be launched separately, or the execution can be interrupted at any time and resumed later from the point of interruption.



If the interrupted execution resumes in another `R` *Session*, the Workspace Setup Section (with all its subsections) of the index.R script **must be run first** at the very beginning of the *Session*.

1.3.1 The Main `R` Script

The index.R script serves as the main orchestrator and *entry point* for *this Project*, focused on Multi-Class Classification (MCC) models applied to the Kaggle Handwritten Character Image Dataset (which uses EMNIST data for Alphabets and Digits). Below is a detailed description of the script's structure, workflow, and core components.



During execution, this script runs several support scripts located in the support-scripts folder. Appendix B of *this Report* describes most of them.

1.3.1.1 Setup & Initialization

The Workspace Setup Section of the index.R code configures and initializes the *Global Environment* for the current `R` session, as well as downloads, prepares, and preprocesses the *project's datasets*.



Code in this Section must run at the very beginning of every *R session* **before any other script executes**.

The code in *this Section* first calls the r-global.path.R script (see Appendix B for details) to initialize the global variables for *this Project*'s essential R scripts directory paths.

Next, in the Project Environment Setup subsection, it executes the setup.R script, which installs the full set of required packages (those not yet installed), loads the required libraries, and completes the remaining initialization.

The remaining operations (carried out in the Preparing Input Data subsection) relate to downloading and preparing the *project datasets*.

1.3.1.2 Downloading & Preprocessing Input Datasets

The code in the Preparing Input Data subsection of the Workspace Setup section downloads and prepares the *project datasets* in the following order:

1. In the Download the Kaggle Dataset subsection, it runs the ds.kaggle.download.R script to retrieve the Kaggle dataset.
2. In the Prepare Training Set subsection, the script prepares the *training datasets* by executing dedicated auxiliary scripts in the following order:
    1. In the Training Set: Load Raw Image Data subsection, it runs the load-raw-image-data.R script to load the *raw image data* of the *Training Set*.
    2. In the Training Set: Prepare Standardized Binary Image Data subsection, it runs the raw-img.dat2std_bin.R script to *standardize and binarize* the raw image data for the *Training Set*.
    3. In the Prepare Training Datasets subsection, it runs the prepare-training-set.R script to split the data of the *Training Set* and prepare *ready to use* datasets for training the *Handwritten Character Recognition (HCR)* models.
3. In the Prepare Final Test Set subsection, the script prepares the *final test datasets* by executing dedicated auxiliary scripts in the following order:
    1. In the Final Test Set: Load Raw Image Data subsection, it runs the load-raw-image-data.R script to load the *raw image data* of the *Validation Set*.
    2. In the Final Test Set: Prepare Standardized Binary Image Data subsection, it runs the raw-img.dat2std_bin.R script to *standardize and binarize* the raw image data for the *Final Test Set*.

1.3.1.3 Machine Learning Model Pipelines

The ML Models Development section implements, tunes, trains, and evaluates a comprehensive benchmarking pipeline for the 39-class Handwritten Character Recognition (HCR) system.

The workflow systematically develops and tests four distinct machine learning architectures, logs evaluation metrics in a comparative results table, and concludes with an in-depth error analysis of common misclassifications.

Additionally, for each model, it generates a visual representation of the following metrics:

- ROC curves;
- Confusion Matrix;
- **A custom metric:** *Per-Class Accuracy (PCA)* (calculated by the user-defined helper function MCClassifier.accuracy.by_class described in section Models Helper / Calculating PCA & Constructing Bar Chart of Appendix A).

1.3.1.3.1 Shallow Learning (SL) Models

The *SL models* are implemented in the following sections of the script:

- The kNN+PCA MCC Model script section implements a model that combines Principal Component Analysis (PCA) (for data preprocessing) and k-Nearest Neighbors (kNN). The implementation, which uses the caret::train function, is described in detail in the kNN+PCA MCC Model section of *this Report*.

- The Random Forest (RF) MCC Model script section implements the Random Forest (RF) algorithm. The implementation, which uses the caret::train function, is described in detail in the Random Forest (RF) MCC Model section of *this Report*.

1.3.1.3.2 Deep Learning (DL) Models

The *DL models* are implemented in the following sections of the script:

- The DNN-Based MCC Model script section implements the Dense Neural Network (DNN) architecture. The implementation, described in detail in the Dense Neural Network Based Multiclass Classifier (DNN-Based MCC) Models section of *this Report*, includes the following two models:
  - *DNN-Based Basic (DNNB) MCC* implemented by the script subsection DNN-Based Basic MCC Model and described in subsection DNN-Based Basic (DNNB) MCC Model;
  - *Fine-Tuned DNN (TDNN) MCC* implemented by the script subsection DNN-Based MCC Model Tuning and described in subsection Tuned DNN-Based (TDNN) MCC Model.
- The CNN-based MCC Model script section implements the Couvolutional Neural Network (CNN) architecture. The implementation, described in detail in the Convolutional Neural Network Based (CNN) MCC Models section of *this Report*, includes the following two models:
  - *CNN-Based Basic (CNNB) MCC* implemented by the script subsection CNN-Based Basic MCC Model and described in subsection CNN-Based Basic (CNNB) MCC Model;
  - *Fine-Tuned CNN (TCNN) MCC* implemented by the script subsection CNN-Based MCC Model Tuning and described in subsection Tuned CNN-Based (TCNN) MCC Model.

1.3.1.3.3 The *Final Test* of the Best Model The latest model we developed (described below in section Tuned CNN-Based (TCNN) MCC Model) achieves the highest accuracy (the metric is used to assess performance). So we apply the *Final Test* to this model using the Validation portion of the *Kaggle dataset*.

The *Final Test* is performed by the 4.tcnn-mcc.final-test.R script invoked from the CNN-Based MCC Model Tuning / Run Scripts section of the *main script*. The TCNN MCC: Final Test section below provides the details.

1.3.1.3.3.1 The Most Frequent Error Results Review The script section Reviewing the Most Frequent Error Results implements the recognition error analysis phase, which pinpoints the most confusing characters that cause the poorest recognition performance.

Finally, it visualizes several image examples of such characters, along with information about which characters they were confused with.

The The Most Confusing Characters for Recognition section below provides the details and visualizations of the confused character examples.

1.4 Datasets Overview

The project uses the HandWritten_Character Dataset downloaded from Kaggle.As mentioned in the section Downloading & Preprocessing Input Datasets above, the dedicated script ds.kaggle.download.R (described below in Appendix B) is used for that purpose. The result of its execution is a folder structure (identical to one shown on the dataset's webpage) containing the raw image data, as shown below:

```
## *** Dataset Raw Data Local Directory Structure ***

## ../../data/raw/Vaibs.HW-Chars
## +-- Train
## |   +-- #
## |   +-- $
## |   +-- &
## |   +-- 0
## |   +-- 1
## |   +-- 2
## |   +-- 3
## |   +-- 4
## |   +-- 5
## |   +-- 6
## |   +-- 7
## |   +-- 8
## |   +-- 9
## |   +-- @
## |   +-- A
## |   +-- B
## |   +-- C
## |   +-- D
## |   +-- E
## |   +-- F
## |   +-- G
## |   +-- H
## |   +-- I
## |   +-- J
## |   +-- K
## |   +-- L
## |   +-- M
## |   +-- N
```

```
## |   +-- P
## |   +-- Q
## |   +-- R
## |   +-- S
## |   +-- T
## |   +-- U
## |   +-- V
## |   +-- W
## |   +-- X
## |   +-- Y
## |   \-- Z
## \-- Validation
##     +-- #
##     +-- $
##     +-- &
##     +-- 0
##     +-- 1
##     +-- 2
##     +-- 3
##     +-- 4
##     +-- 5
##     +-- 6
##     +-- 7
##     +-- 8
##     +-- 9
##     +-- @
##     +-- A
##     +-- B
##     +-- C
##     +-- D
##     +-- E
##     +-- F
##     +-- G
##     +-- H
##     +-- I
##     +-- J
##     +-- K
##     +-- L
##     +-- M
##     +-- N
##     +-- P
##     +-- Q
##     +-- R
##     +-- S
##     +-- T
##     +-- U
```

```
##      +-- V
##      +-- W
##      +-- X
##      +-- Y
##      \-- Z
```

As described on the dataset's landing page, the *context* of the dataset is as follows:

> 1. Used *EMNIST* data for Alphabets and Digits.
> 2. Transformed the Data using some image processing techniques and convert it to 32,32 pixel black and white images.
> 3. Created the Data set for special character (@, #, $, &)
> 4. Merged the Categories to avoid the misclassification
> 5. Total 39 Categories in Train and Validation set

However, although the dataset author claims that the images were transformed into 32x32-pixel grayscale images, a quick analysis of the raw data showed that at least some images have different sizes.

So the author of *this Report* has decided to standardize the images by resizing them to the smallest image size found in the dataset.

This functionality has been implemented in the raw-img.dat2std_bin.R support script described below in Appendix B: Support Scripts.

First, the Define the Standard Image Size Geometry section of the script implements the *Dynamic Geometry Determination* feature, which determines the geometry parameters for the image standardization..

Execution of this section yields the following result:

```
## The geometry for standardizing images to a uniform size,
## based on the smallest existing image in the Training Set, is as follows:
## 28x28!
```

Meaning that the smallest image size is 28x28 pixels.

Figure 1.1 below shows a few sample images, both 28x28 and 32x32 pixels.

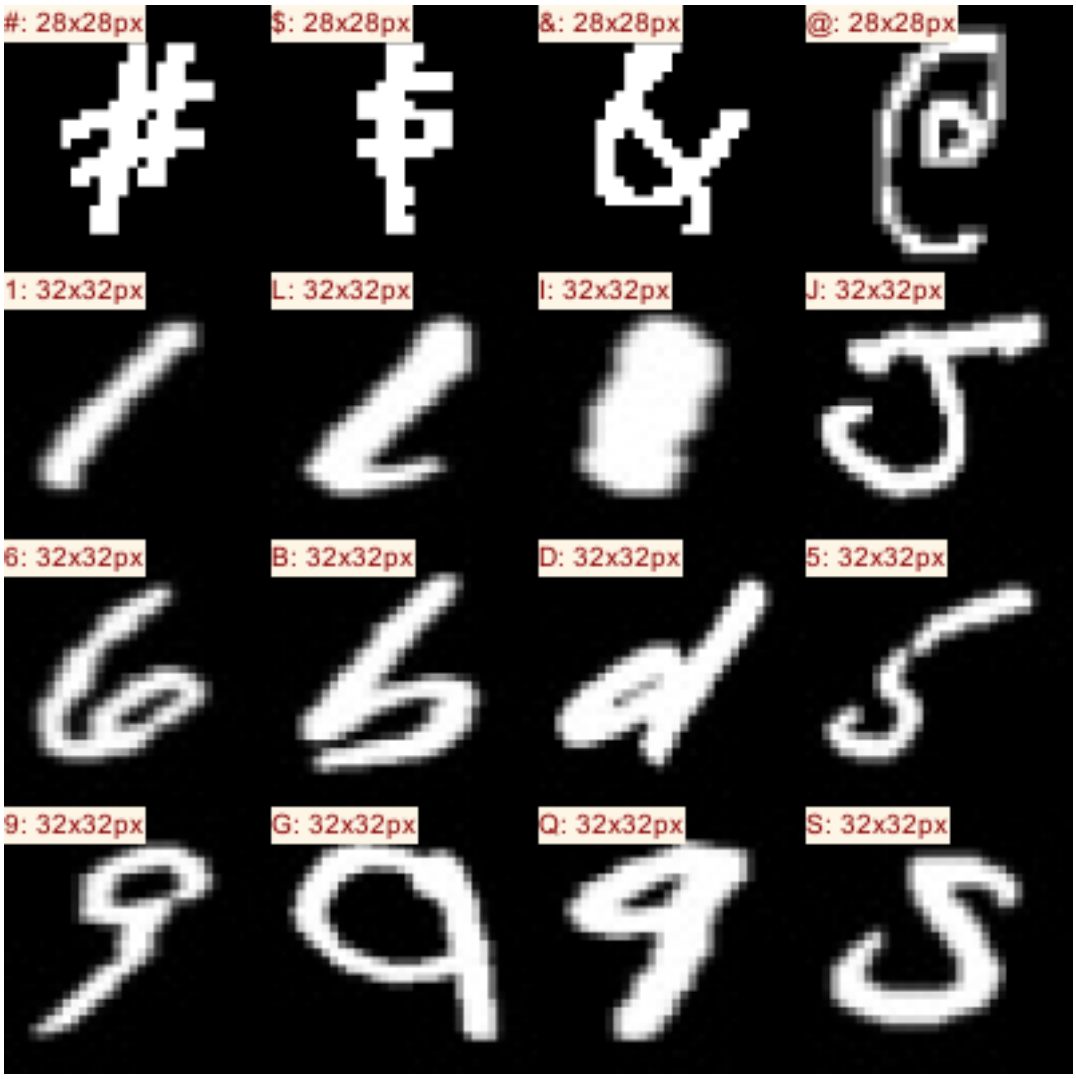


Figure 1.1: Example images, both 28x28 and 32x32 pixels.

Chapter 2

Methods and Analysis

2.1 kNN+PCA MCC Model

2.2 Random Forest (RF) MCC Model

Random Forests (RF) approach explained in [Section 30.5 Random forests] of the *Course Textbook (New Edition)*[6]

2.3 Dense Neural Network Based Multiclass Classifier (DNN-Based MCC) Models

2.3.1 DNN-Based Basic (DNNB) MCC Model

2.4 Tuned DNN-Based (TDNN) MCC Model

2.5 Convolutional Neural Network Based (CNN) MCC Models

2.5.1 CNN-Based Basic (CNNB) MCC Model

2.6 Tuned CNN-Based (TCNN) MCC Model

2.6.1 TCNN MCC: Final Test

Chapter 3

Results

3.1 The Most Confusing Characters for Recognition

Chapter 4

Conclusion

Chapter 5

Appendix A: Support Functions

5.1 Data Helper Script

5.1.1 Data Loading



The complete source code of the functions described in this Section is available in the the Data Loading Section of the data-helper.R script on *GitHub*.

5.1.1.1 kaggle_cli.download Function

Downloads datasets from Kaggle into a specified local directory using the Kaggle command-line interface* (CLI).

5.1.1.1.1 Usage

```
kaggle_dataset <- "vaibhao/handwritten-characters"

  put_log("Downloading dataset `%1` ...", kaggle_dataset)
  kaggle_cli.download(kaggle_dataset, raw_data.chars.dir, unzip = TRUE)
```

```
## The Kaggle image files have been downloaded
## and unziped to the following directory:
## '../../data/raw/Vaibs.HW-Chars'

## *** Dataset Raw Data Local Directory Structure ***

## ../../data/raw/Vaibs.HW-Chars
## +-- Train
## |   +-- #
## |   +-- $
## |   +-- &
## |   +-- 0
## |   +-- 1
## |   +-- 2
## |   +-- 3
## |   +-- 4
## |   +-- 5
## |   +-- 6
## |   +-- 7
## |   +-- 8
## |   +-- 9
## |   +-- @
## |   +-- A
## |   +-- B
## |   +-- C
## |   +-- D
## |   +-- E
## |   +-- F
## |   +-- G
## |   +-- H
## |   +-- I
## |   +-- J
## |   +-- K
## |   +-- L
## |   +-- M
## |   +-- N
## |   +-- P
```

```
## |   +-- Q
## |   +-- R
## |   +-- S
## |   +-- T
## |   +-- U
## |   +-- V
## |   +-- W
## |   +-- X
## |   +-- Y
## |   \-- Z
## \-- Validation
##     +-- #
##     +-- $
##     +-- &
##     +-- 0
##     +-- 1
##     +-- 2
##     +-- 3
##     +-- 4
##     +-- 5
##     +-- 6
##     +-- 7
##     +-- 8
##     +-- 9
##     +-- @
##     +-- A
##     +-- B
##     +-- C
##     +-- D
##     +-- E
##     +-- F
##     +-- G
##     +-- H
##     +-- I
##     +-- J
##     +-- K
##     +-- L
##     +-- M
##     +-- N
##     +-- P
##     +-- Q
##     +-- R
##     +-- S
##     +-- T
##     +-- U
##     +-- V
```

```
##     +-- W
##     +-- X
##     +-- Y
##     \-- Z
```

5.1.1.1.2 Arguments

- **dataset.dir:** *Remote path* to the Kaggle dataset;
- **data.local_path:** *Local directory path* where the dataset files are saved;
- **unzip:** (Optional, *Boolean*, $FALSE$ by default) Logical flag indicating whether the *downloaded archive* should be automatically unzipped.

5.1.1.1.3 Details

Below is the *step-by-step execution flow* of the function:

1. **Prerequisite Check:** Verifies that the Kaggle CLI is installed and accessible in the system's PATH (by running the kaggle –version command).
2. **Download Operation:** Executes the CLI command `kaggle datasets download <kaggle_dataset> --path <path>` (with an optional `--unzip` flag if requested).
3. **Verification:** Checks that the local destination directory exists after the download completes, indicating that the download operation was successful.

5.1.1.1.4 Source Code

The source code of the function is shown below:

```
kaggle_cli.download <- function(dataset.dir, data.local_path, unzip = FALSE) {
  # Check for Kaggle CLI Installation
  if (system("kaggle --version", ignore.stdout = TRUE, ignore.stderr = TRUE) != 0) {
    #' Kaggle command-line interface is missing or not configured
    #' in your system's PATH.
    stop("Kaggle CLI is not installed or not in the PATH.")
  }

  # Download the Dataset via CLI
  if (system(trimws(paste("kaggle datasets download", kaggle_dataset, "--path",
                          data.local_path, ifelse(unzip, "--unzip", "")))) != 0) {
    stop(str.build("Failed to download the dataset with Kaggle CLI: `%1`.", dataset.dir))
  }

  #'Verify that the dataset's local directory exists,
  #'as it should have been created automatically during the download.
  if(!dir.exists(data.local_path)) {
    stop(str.build("Failed to download and unzip the Kaggle dataset: `%1`.", dataset.dir))
  }
}
```

5.1.1.2 load_image.list Function

This is a core utility designed to recursively traverse a root directory passed in the root_path argument, discover character-specific (or class-specific, in the terms of the project's multiclass classifier models) subfolders, load data from raw image files (downloaded from the Kaggle platform) using a custom parsing function, the pointer to which is passed in the fn.load_image.dat argument, filter out empty or invalid images, and organize everything into an R list object with the corresponding labels and file paths.

5.1.1.2.1 Usage



In the *code snippet* below, a pointer to the auxiliary function load.img_trimmed.dat described below in *this Section* is used as the value of the argument fn.load_image.dat.

```
  put_log("Creating an Image list from the raw data files
stored in the following root directory: %1,
Please wait...", img.train_root.dir)

  train.raw_img.dat.ls <- load_image.list(root_path = img.train_root.dir,
                                          fn.load_image.dat = load.img_trimmed.dat
                                          # folder.list = NULL,
                                          # char_files.max = NA,
                                          # seed = NA,
                                          # random_sample = FALSE
                                          )
```

```
## *** Training Image Data List Structure ***

## Class Labels:
##  [1] # $ & @ 0 1 2 3 4 5 6 7 8 9 A B C D E F G H I J K L M N P Q R S T U V W X Y Z
## Levels: # $ & @ 0 1 2 3 4 5 6 7 8 9 A B C D E F G H I J K L M N P Q R S T U V W X Y Z

## Class Name: #
## Image Files Number: 15600
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/#/__0_1004356.png
## Image File Shape:
## shape(1, 28, 28)
```

```
##
## Class Name: $
## Image Files Number: 16199
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/$/__0_1011576.png
## Image File Shape:
## shape(1, 28, 28)
##
## Class Name: &
## Image Files Number: 13000
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/&/__0_1027442.png
## Image File Shape:
## shape(1, 28, 28)
##
## Class Name: @
## Image Files Number: 38009
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/@/___0_2818202.png
## Image File Shape:
## shape(1, 28, 28)
##
## Class Name: 0
## Image Files Number: 65504
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/0/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 1
## Image Files Number: 43773
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/1/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 2
## Image Files Number: 39351
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/2/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 3
## Image Files Number: 39996
## Image File Path example:
```

```
## data/raw/Vaibs.HW-Chars/Train/3/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 4
## Image Files Number: 38112
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/4/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 5
## Image Files Number: 32317
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/5/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 6
## Image Files Number: 38879
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/6/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 7
## Image Files Number: 41080
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/7/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 8
## Image Files Number: 38795
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/8/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: 9
## Image Files Number: 38319
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/9/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
```

```
## Class Name: A
## Image Files Number: 17205
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/A/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: B
## Image Files Number: 8666
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/B/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: C
## Image Files Number: 13560
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/C/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: D
## Image Files Number: 15509
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/D/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: E
## Image Files Number: 32627
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/E/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: F
## Image Files Number: 11635
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/F/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: G
## Image Files Number: 5443
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/G/_1_0.jpg
```

```
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: H
## Image Files Number: 12133
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/H/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: I
## Image Files Number: 13873
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/I/_1_1000.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: J
## Image Files Number: 4261
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/J/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: K
## Image Files Number: 4334
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/K/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: L
## Image Files Number: 21648
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/L/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: M
## Image Files Number: 12089
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/M/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: N
```

```
## Image Files Number: 21421
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/N/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: P
## Image Files Number: 11095
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/P/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: Q
## Image Files Number: 4707
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/Q/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: R
## Image Files Number: 20498
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/R/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: S
## Image Files Number: 25910
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/S/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: T
## Image Files Number: 30853
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/T/_1_0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: U
## Image Files Number: 16385
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/U/0.jpg
## Image File Shape:
```

```
## shape(1, 32, 32)
##
## Class Name: V
## Image Files Number: 7246
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/V/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: W
## Image Files Number: 7266
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/W/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: X
## Image Files Number: 5106
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/X/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: Y
## Image Files Number: 6762
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/Y/0.jpg
## Image File Shape:
## shape(1, 32, 32)
##
## Class Name: Z
## Image Files Number: 4866
## Image File Path example:
## data/raw/Vaibs.HW-Chars/Train/Z/1.jpg
## Image File Shape:
## shape(1, 32, 32)
```

5.1.1.2.2 Arguments

- **root_path:** The root directory path containing subfolders of images organized by character recognition classes;

- **fn.load_image.dat:** Pointer to a user-defined loading and processing helper function applied to each image file path (e.g., a function to read image matrices or binary arrays);

- **folder.list:** (Optional, *Character*, `NULL` by default) Vector of specific subfolder names to process. If `NULL`, it defaults to all directories found inside the *root path* passed in the root_path argument;

- **char_files.max:** (Optional, *Integer*, `NA` by default) Numeric value limiting the maximum number of files to load per folder (useful for subsampling or testing);

- **seed:** (Optional, *Integer*, `NA` by default) Random seed used for reproducible sampling when char_files.max is specified (ignored otherwise);

- **random_sample:** (Optional, *Boolean*, `FALSE` by default) Boolean (`TRUE/FALSE`) indicating whether files within a folder should be sampled randomly (`TRUE`) or taken sequentially from the top (`FALSE`).

5.1.1.2.3 Details

Below is the *step-by-step execution flow* of the function:

1. **File Path Discovery:** Internally invokes the img.file.get_list user-defined helper function (described below in the Image Processing Section) to gather a structured list of file paths from each target subfolder, applying optional maximum limits (passed in the char_files.max argument), random sampling (depending on the value passed in the random_sample argument), and seed generation (if the seed argument value is not `NA`).
2. **Label Generation:** Extracts directory names from the file path list and converts them into a factor to serve as classification labels.

3. **Iterative processing of class directories:** Iterates through each directory, performing the following operations:

    a) **Batch Loading:** Calls the custom loader function (the pointer to which is passed in the fn.load_image.dat argument) using the map `R` function across all file paths in the folder.
    b) **Data Validation & Filtering:** Evaluates a validity check on the loaded image data. Specifically, it checks that the binary matrix elements sum to greater than zero, effectively filtering out blank, corrupt, or unreadable images.

4. **Consistency Checks:** Enforces a safety assertion to ensure that the filtered image list length strictly matches the corresponding filtered file path list length.

5.1.1.2.4 Value

Structured list containing two primary components:

1. *Label list:* A factor of *class labels*;
2. *Image list:* A nested list mapping each *class folder* to its valid loaded *image data* and corresponding *file paths*.

5.1.1.2.5 Source Code

The simplified version of the function's source code is shown below:

```
load_image.list <- function(root_path,
                            fn.load_image.dat,
                            folder.list = NULL,
                            char_files.max = NA,
                            seed = NA,
                            random_sample = FALSE) {
  #'Gather a structured list of file paths from each target subfolder,
  #'applying optional maximum limits, random sampling, and seed generation.
  img.file_list <- img.file.get_list(root_path,
                                     folder.list,
                                     char_files.max,
                                     seed,
                                     random_sample)

  #' Extracts directory names from the file path list
  #' and converts them into a factor to serve as classification labels.
  label_list <- as.factor(names(img.file_list))

  # Load structured image data list
  img_list <- lapply(img.file_list, function(char.dir){

    #' Batch Loading using the custom loader function `fn.load_image.dat`.
    img_dat.ls <- map(char.dir$file_path.list,
                      fn.load_image.dat)

    # Evaluates a validity check on the loaded image data.
    valid.idx <- sapply(img_dat.ls, function(img.dat){
      mx.bin <- img.dat[1,,]
      mode(mx.bin) <- "numeric"

      # Filter out blank, corrupt, or unreadable images.
      sum(mx.bin) > 0
    })

    ls <- list(img.list = img_dat.ls[valid.idx],
               fpath.list = char.dir$file_path.list[valid.idx])

    #' Ensures that the filtered image list length strictly matches
    #' the corresponding filtered file path list length.
    stopifnot(length(ls$img.list) == length(ls$fpath.list))
```

```
    ls
  })

  list(label.list = label_list,
       img.list = img_list)
}
```



The complete version of the source code is available in the Data Loading Section of the data.helper.R script on *GitHub*.

5.1.1.3 img.file.get_list Function

This function traverses *image directories*, collects *file paths* grouped by *folder* (representing *class labels*), and optionally limits, seeds, or randomly samples files for *dataset preparation*.

5.1.1.3.1 Usage



In the *code snippet* below, *this function* is called internally from the load_image.list function described above in *this Section*.

```
put_log("Function `load_image.list`:
Getting file path lists from the following Root Path:
%1", root_path)
```

```
## Function `load_image.list`:
## Getting file path lists from the following Root Path:
## ../../data/raw/Vaibs.HW-Chars/Train
```

```
folder.list <- NULL
char_files.max <- NA
seed <- NA
random_sample <- FALSE


#'Gather a structured list of file paths from each target subfolder,
#'applying optional maximum limits, random sampling, and seed generation.
img.file_list <- img.file.get_list(root_path,
                                   folder.list,
                                   char_files.max,
                                   seed,
                                   random_sample)
```

```
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/#
## Function: `img.file.get_list`:
## 15600 files in folder: #
## Function: `img.file.get_list`:
```

```
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/$
## Function: `img.file.get_list`:
## 16199 files in folder: $
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/&
## Function: `img.file.get_list`:
## 13000 files in folder: &
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/@
## Function: `img.file.get_list`:
## 38009 files in folder: @
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/0
## Function: `img.file.get_list`:
## 65504 files in folder: 0
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/1
## Function: `img.file.get_list`:
## 43773 files in folder: 1
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/2
## Function: `img.file.get_list`:
## 39351 files in folder: 2
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/3
## Function: `img.file.get_list`:
## 39996 files in folder: 3
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/4
## Function: `img.file.get_list`:
## 38112 files in folder: 4
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/5
## Function: `img.file.get_list`:
## 32317 files in folder: 5
## Function: `img.file.get_list`:
## Getting file path list from the following char's root folders:
```

```
## ../../data/raw/Vaibs.HW-Chars/Train/6
## Function: ‘img.file.get_list‘:
## 38879 files in folder: 6
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/7
## Function: ‘img.file.get_list‘:
## 41080 files in folder: 7
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/8
## Function: ‘img.file.get_list‘:
## 38795 files in folder: 8
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/9
## Function: ‘img.file.get_list‘:
## 38319 files in folder: 9
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/A
## Function: ‘img.file.get_list‘:
## 17205 files in folder: A
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/B
## Function: ‘img.file.get_list‘:
## 8666 files in folder: B
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/C
## Function: ‘img.file.get_list‘:
## 13560 files in folder: C
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/D
## Function: ‘img.file.get_list‘:
## 15509 files in folder: D
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/E
## Function: ‘img.file.get_list‘:
## 32627 files in folder: E
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/F
```

```
## Function: ‘img.file.get_list‘:
## 11636 files in folder: F
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/G
## Function: ‘img.file.get_list‘:
## 5443 files in folder: G
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/H
## Function: ‘img.file.get_list‘:
## 12134 files in folder: H
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/I
## Function: ‘img.file.get_list‘:
## 13873 files in folder: I
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/J
## Function: ‘img.file.get_list‘:
## 4261 files in folder: J
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/K
## Function: ‘img.file.get_list‘:
## 4334 files in folder: K
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/L
## Function: ‘img.file.get_list‘:
## 21648 files in folder: L
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/M
## Function: ‘img.file.get_list‘:
## 12089 files in folder: M
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/N
## Function: ‘img.file.get_list‘:
## 21421 files in folder: N
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/P
## Function: ‘img.file.get_list‘:
```

```
## 11095 files in folder: P
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/Q
## Function: ‘img.file.get_list‘:
## 4707 files in folder: Q
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/R
## Function: ‘img.file.get_list‘:
## 20498 files in folder: R
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/S
## Function: ‘img.file.get_list‘:
## 25911 files in folder: S
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/T
## Function: ‘img.file.get_list‘:
## 30853 files in folder: T
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/U
## Function: ‘img.file.get_list‘:
## 16385 files in folder: U
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/V
## Function: ‘img.file.get_list‘:
## 7246 files in folder: V
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/W
## Function: ‘img.file.get_list‘:
## 7266 files in folder: W
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/X
## Function: ‘img.file.get_list‘:
## 5106 files in folder: X
## Function: ‘img.file.get_list‘:
## Getting file path list from the following char’s root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/Y
## Function: ‘img.file.get_list‘:
## 6762 files in folder: Y
```

```
## Function: 'img.file.get_list':
## Getting file path list from the following char's root folders:
## ../../data/raw/Vaibs.HW-Chars/Train/Z
## Function: 'img.file.get_list':
## 4867 files in folder: Z
```

5.1.1.3.2 Arguments

- **root_path:** The root directory path containing subfolders of images organized by character recognition classes;
- **folder.list:** (Optional, *Character*, `NULL` by default) Vector of specific subfolder names to process. If `NULL`, it defaults to all directories found inside the *root path* passed in the root_path argument;
- **char_files.max:** (Optional, *Integer*, `NA` by default) Numeric value limiting the maximum number of files to list per folder (useful for subsampling or testing);
- **seed:** (Optional, *Integer*, `NA` by default) Random seed used for reproducible sampling when char_files.max is specified (ignored otherwise);
- **random_sample:** (Optional, *Boolean*, `FALSE` by default) Logical ($TRUE/FALSE$) value indicating whether files within a folder should be sampled randomly (`TRUE`) or taken sequentially from the top (`FALSE`).

5.1.1.3.3 Details

This is an auxiliary function that is used internally in the load_image.list function as shown above in the Usage Section of *this Description*.

Below is the *step-by-step execution flow* of the function:

1. **Subfolder Discovery:** If the folder.list argument value is `NULL` (the *folder list* is not provided), the function retrieves all subdirectory names inside the *root directory* (using the path passed in the root_path argument) and uses them as the *folder list* for further processing. Otherwise, the folder.list value is used.

2. **File Path Collection:** Next, it iterates through the *folder list* determined in the previous step, constructs the *full path* for each folder, and retrieves all file paths in that folder.
3. **Capping and Sampling:** If the value for the char_files.max argument is provided and does not exceed the folder's file count, the number of files to process is limited to that value.



- If the condition above is met, the *file selection method* depends on the value of the random_sample argument. If it is `TRUE`, a *sample* of the size provided in the char_files.max argument is taken using a *reproducibility seed offset* by the folder's index if a *seed* is provided (the value of the seed argument is not `NA`). Otherwise, a top list of the size provided in the char_files.max argument is selected.
- Otherwise, the full list of file paths obtained in the previous step is used for the final result.

5.1.1.3.4 Value

A *named nested list* where each element represents a *subfolder*, containing its *root path* and the *file path list*.

5.1.1.3.5 Source Code

The slightly simplified version of the function's source code is shown below:

```
img.file.get_list <- function(root_path,
                                    folder.list = NULL,
                                    char_files.max = NA,
                                    seed = NA,
                                    random_sample = FALSE) {
  if(is.null(folder.list))
    # Scans the root path to find all available subfolders.
    folder.list <- dir(root_path)

  # Generates a file path list
  path_list <- lapply(folder.list, function(folder_name){
    #' Constructs the full path and retrieves all file paths
    #' for the given folder name.
```

```
    file.root_path <- file.path(root_path, folder_name)
    folder.idx <- which(folder.list == folder_name)

    fpath.list <- list.files(file.root_path,
                             full.names = TRUE)

    fpath.len <- length(fpath.list)

    # If the number of files is limited and
    # the total number of files in the folder exceeds the limit
    if(!is.na(char_files.max) && fpath.len > char_files.max){

      # If a seed is provided.
      if (!is.na(seed)) {
        #' Applies a reproducibility seed offset
        #' by the folder's index
        set.seed(seed + folder.idx)
      }


      if(random_sample) {
        # Samples the files randomly
        file.idx <- sample(fpath.len, size = char_files.max)
      } else {
        #' Selects the `char_files.max` top files
        #' according to the specified limit.
        file.idx <- seq_len(char_files.max)
      }

      fpath.list <- fpath.list[file.idx]
    }

    list(root_path = file.root_path,
         file_path.list = fpath.list)
  })

  #' Shortens the name of each list item to the first character
  #' of the folder name..
  names(path_list) <- folder.list |> substr(1,1)
  path_list
}
```



The complete version of the source code is available in the Data Loading Section of the data.helper.R script on *GitHub*.

5.1.1.4 load.img_trimmed.dat Function

This specialized utility function reads an image file, dynamically detects whether it contains nonzero pixel data, optionally trims whitespace or empty borders around the visual features using the magick package, and extracts its raw data matrix structure.

5.1.1.4.1 Usage

```
train.raw_img.dat.ls <- load_image.list(root_path = img.train_root.dir,
                                        fn.load_image.dat = load.img_trimmed.dat
                                        # folder.list = NULL,
                                        # char_files.max = NA,
                                        # seed = NA,
                                        # random_sample = FALSE
                                        )
```



In the *code snippet* above, a pointer to *this function* is passed in the fn.load_image.dat argument to the load_image.list function described above in *this Section*.

5.1.1.4.2 Arguments

- **file_path:** Path to the image file to load.

5.1.1.4.3 Details

Below is the *step-by-step execution flow* of the function:

1. ***Image Loading:*** The function begins by reading the image file from disk into memory as an object using magick::image_read.
2. ***Matrix Conversion & Validation:*** Converts the loaded image into a numeric matrix representation using the user-defined helper function magick_img2matrix described below in described below in the Image Processing Section.

3. ***Image Validation & Conditional Cropping:*** Evaluates the matrix's maximum value to check whether the image is entirely blank or empty. If valid content exists (the maximum pixel value is greater than zero), it applies magick::image_trim to automatically crop out uniform background borders or whitespace, centering the content around the core visual features.

4. ***Data Extraction:*** Finally, it extracts and returns the underlying raw pixel buffer data using the magick::image_data helper function.



This is an auxiliary function, the pointer to which is passed in the fn.load_image.dat argument to the load_image.list function, as shown above in the Usage Section of *this Description*.

5.1.1.4.4 Value

Raw pixel buffer data from the trimmed image, returned by the internal call to magick::image_data.

5.1.1.4.5 Source Code

The source code of the function is shown below:

```
load.img_trimmed.dat <- function(file_path) {
  # Reads the image file from disk
  img <- magick::image_read(file_path)

  # Converts the loaded image into a numeric matrix representation
  img.mx <- magick_img2matrix(img)

  img.trimmed <- img

  #' Evaluates the matrix's maximum value to check
  #' whether the image is entirely blank or empty.
  if (max(img.mx) > 0) {
    img.trimmed <- image_trim(img)
  }

  # Extracts and returns the underlying raw pixel buffer data.
  magick::image_data(img.trimmed)
}
```



During the execution, it calls internally the user-defined helper function magick_img2matrix described below in Section Image Processing of Appendix A of *this Report*.

5.1.2 Image Processing



The complete source code of the functions described in this Section is available in the Image Processing Section of the data-helper.R script on *GitHub*.

5.1.2.1 magick_img2matrix Function

This is a lightweight utility that converts a ***magick*** *image* object into a *normalized numeric matrix of pixel intensities.*

5.1.2.1.1 Usage

```
# Sample image file:
file_path <- "../../data/raw/Vaibs.HW-Chars/Train/W/0.jpg"
img <- magick::image_read(file_path)
```

```
## *** Image visualization before processing ***
```

```
# Reads the image file from disk
plot(img)
```



In the *code snippet* below, *this function* is called internally from the load.img_trimmed.dat function described above in Section Data Loading.

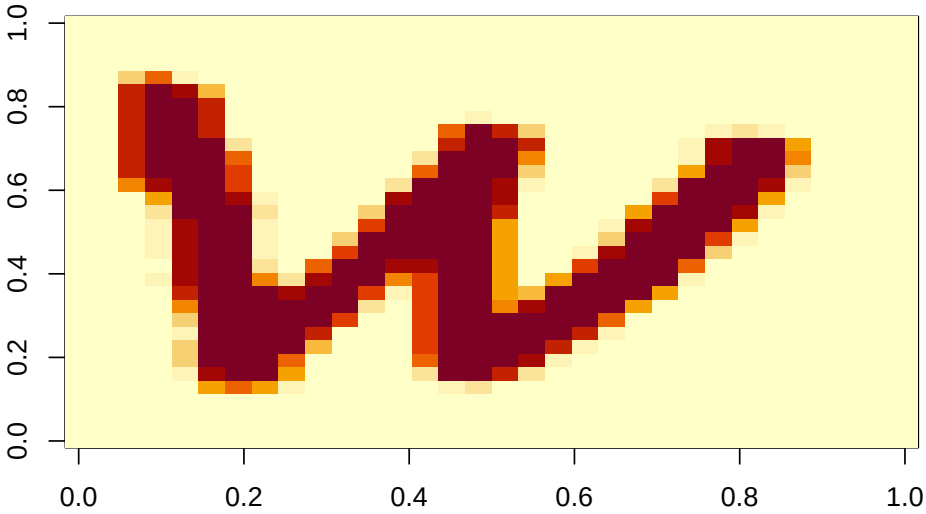
```
# Converts the loaded image into a numeric matrix representation
img.mx <- magick_img2matrix(img)
```

```
## *** Image data view after processing ***
```

```
str(img.mx)
```

```
##  num [1:32, 1:32] 0 0.00392 0 0 0.00784 ...
```

```
image.mx(img.mx)
```



5.1.2.1.2 Arguments

- **img:** A Magick Image object (typically processed, resized, or filtered prior to conversion).

5.1.2.1.3 Details

Below is the *step-by-step execution flow* of the function:

1. ***Raw Binary Data Extraction:*** The function calls the [image_data] function to extract the underlying raw hexadecimal or integer byte array buffer from the *magick* Image object.
2. ***Color Channel Isolation:*** Slices the multidimensional array to isolate the primary color channel matrix. For grayscale or black-and-white pipeline images (as in our case), this captures the single intensity plane.
3. ***Data Type Conversion:*** Explicitly casts the extracted data array elements from raw/character bytes into a numeric data type, making mathematical operations possible.
4. ***Min-Max Normalization:*** Divides the entire matrix by 255 (the maximum 8-bit color intensity value) to normalize pixel values into a clean floating-point scale ranging from 0.0 (black/background) to 1.0



This is a utility function used internally by the other user-defined functions, as shown in the usage example above in the Usage Section of *this Description*.

5.1.2.1.4 Value

Grayscale image data matrix representing *pixel intensity*, normalized to a *floating-point* scale ranging from 0.0 (black/background) to 1.0.

5.1.2.1.5 Source Code

The source code of the function is shown below:

```
magick_img2matrix <- function(img){
  #' Extract the underlying raw byte array buffer
  #' from the `magick` image object.
  img.dat <- image_data(img)

  #' Slice the multidimensional array to isolate
  #' the primary color channel matrix (capture the single intensity plane).
  mx.bin <- img.dat[1,,]

  #' Cast the extracted data array elements from raw/character bytes
  #' into a numeric data type.
  mode(mx.bin) <- "numeric"

  #' Normalize pixel values into a clean floating-point scale
  #' ranging from 0.0 (black/background) to 1.0.
```

```
  mx.bin /255
}
```

5.2 Models Helper

5.2.1 Calculating PCA & Constructing Bar Chart

5.2.1.1 MCClassifier.accuracy.by_class Function

5.2.1.1.1 Usage



In the *code snippet* above, *this function* is called internally from the outer_MCClassifier.accuracy.by_class function described above in *this Section*.
In the *code snippet* above, a pointer to *this function* is passed in the fn.load_image.dat argument to the load_image.list function described above in *this Section*.

5.2.1.1.2 Arguments

- **arg1:** ;
- **arg2:** (Optional, *Numeric*, 0 by default) ;
- **arg3:** (Optional, *Numeric*, `NA` by default) ;
- **arg4:** (Optional, *Integer*, `NA` by default) ;
- **arg5:** (Optional, *Boolean*, `FALSE` by default) ;
- **arg6:** (Optional, *Character*, `NULL` by default) ;

5.2.1.1.3 Details



This is an auxiliary function that is used internally in the function as shown above in the Usage Section of *this Description*.

5.2.1.1.4 Value

5.2.1.1.5 Source Code

The source code of the function is shown below:

The simplified version of the function's source code is shown below:



The source codeis also available in the Image Processing Section of the data.helper.R script on *GitHub*.
The complete version of the source code is available in the Image Processing Section of the data.helper.R script on *GitHub*.

Chapter 6

Appendix B: Support Scripts

6.1 r-global.path.R Script

This script establishes and validates the following paramount `R` scripts' directory paths:

- root `R` scripts' directory;
- support scripts' directory;
- user-defined support (helper) functions' scripts directory.

6.2 setup.R Script

This script serves as the master initialization and environment-configuration script for the entire project. Its primary purpose is to set up and configure all software dependencies, Python environments, directory paths, and helper functions before running any project execution scripts.

The first two lines of the code perform the following operations in order:

1. Adjust the global download timeout to handle packages and large dataset downloads;
2. Increase the expression nesting limit.

Below is a detailed breakdown of the code by sections.

6.2.1 Install Packages Section

The code in this Section automatically checks for required `R` packages across multiple domains and installs them if some or all are not installed.

6.2.2 Load Libraries & resolve conflicts Section

6.2.3 Load Logging Helper Functions Section

6.2.4 Init Project Directories Section

6.2.5 Init Input Data Paths Section

6.2.6 Init Project Script Paths Section

6.2.7 Load Common Helper Functions Section

6.2.8 Load Data Helper Functions Section

6.2.9 Load Model Helper Functions Section

6.2.10 Load `CNN_MCC.HyperModel` helper class Section

6.2.11 Load `CNN_MCC.HyperParameters` helper class Section

6.3 ds.kaggle.download.R Script

The script automates downloading and unpacking the *39-class Handwritten Characters (HWC)* dataset from Kaggle directly into the project's `R` *Environment Workspace*.

The download process consists of the following two steps:

1. Conditionally downloads and unzips the *Kaggle Dataset (KDS)* using the user-defined helper function kaggle_cli.download described above in Appendix A, if the local directory, whose path is passed to the function in the data.local_path argument, does not exist.
2. Conditionally cleans up the local dataset directory structure by safely deleting the duplicated subdirectory, if it exists.

6.4 load-raw-image-data.R Script

6.5 raw-img.dat2std_bin.R Script

6.6 prepare-training-set.R Script

6.7 Script

6.8 Script

6.9 Script

6.10 Script

Chapter 7

Appendix C: Device (Laptop) Used for Development & Testing

7.1 Ninkear Laptop

7.1.1 System

7.1.1.1 Device Specifications

- **Processor:** 13th Gen Intel(R) Core(TM) i7-13620H (2.40 GHz)
- **Installed RAM:** 32.0 GB (31.7 GB usable)
- **System type:** 64-bit operating system, x64-based processor

7.1.1.2 Windows Specifications

- **Edition:** Windows 11 Pro
- **Version:** 25H2
- **Installed on:** 12/14/2024
- **OS build:** 26200.8973
- **Experience:** Windows Feature Experience Pack 1000.26100.344.0

Bibliography

[1] Rafael A. Irizarry. *Introduction to Data Science, Part I. Data Wrangling and Visualization with R*. Oct. 24, 2019. URL: https://rafalab.dfci.harvard.edu/dsbook-part-1/ (cit. on p. 5).

[2] Rafael A. Irizarry. *Introduction to Data Science, Part II. Statistics and Prediction Algorithms Through Case Studies*. Dec. 27, 2024. URL: https://rafalab.dfci.harvard.edu/dsbook-part-2/ (visited on 02/18/2025) (cit. on p. 5).

[3] Ajay Agarwal, Sartaj Ahmad, and Minakshi. "Handwritten Character Recognition: A Comprehensive Survey". In: *2023 Second International Conference on Informatics (ICI)*. 2023, pp. 1–6. DOI: 10.1109/ICI60088.2023.10420981. URL: https://ieeexplore.ieee.org/document/10420981 (cit. on p. 6).

[4] Azamat Kurbanaev. *edX Data Science: Capstone, Choose Your Own. Handwritten Character Recognition Multiclass Classifier*. Version 1.0.0. Oct. 5, 2026. URL: https://github.com/AzKurban-edX-DS/edx.capstone.choose-your-own (cit. on p. 6).

[5] Tahir Cetin Akinci, Oguzhan Topsakal, and M. Ilhan Akbas. "Machine Learning Methods from Shallow Learning to Deep Learning". In: *Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions*. Ed. by Ömer Faruk Ertuğrul, Josep M. Guerrero, and Musa Yilmaz. Cham: Springer Nature Switzerland, 2024, pp. 1–28. ISBN: 978-3-031-69499-8. DOI: 10.1007/978-3-031-69499-8_1. URL: https://doi.org/10.1007/978-3-031-69499-8_1 (cit. on p. 7).

[6] Rafael A. Irizarry. *Introduction to Data Science, Part II, Chapter 30, Section 30.5: Random forests. Statistics and Prediction Algorithms Through Case Studies*. Dec. 27, 2024. URL: https://rafalab.dfci.harvard.edu/dsbook-part-2/ml/algorithms.html#sec-random-forests (visited on 02/18/2025) (cit. on p. 18).